

Name: Wilbert Chandra

NIM: 2602113666

1. Explain what LSC is!

Longest Common Subsequence (LCS): LCS is a classic computer science problem that finds the longest subsequence common to all sequences in a set of sequences (often just two sequences). It differs from the longest common substring: unlike substrings, subsequences are not required to occupy consecutive positions within the original sequences¹. The problem of computing longest common subsequence is the basis of data comparison programs such as the diff utility and has applications in computational linguistics and bioinformatics.

2. You can solve LSC problem with DP, is it possible with greedy method? If it is possible, explain your algorithm!

Solving LCS with Dynamic Programming (DP) and Greedy Method: The LCS problem is typically solved using dynamic programming. However, a greedy approach may not always yield the correct result for the LCS problem. This is because a greedy algorithm makes the locally optimal choice at each stage with the hope of finding the global optimum, but the LCS problem does not always satisfy this property. In other words, a locally optimal choice may lead to a sub-optimal solution in the case of the LCS problem.

3. In this life, give an example of LSC case!

Real-life example of LCS: A real-life example of LCS can be found in the field of text comparison or diffing tools. For instance, version control systems like Git use LCS algorithms to determine the differences between different versions of source code files. Another example could be in DNA sequence alignment in bioinformatics, where scientists look for similar patterns or sequences across different DNA strands.

4. Give the pseudocode to reconstruct an LCS from only c table in $O(m+n)$ time without using the b table.

```
PRINT-LCS(c, X, Y, i, j)

if c[i, j] == 0
    return

if X[i] == Y[j]
    PRINT-LCS(c, X, Y, i - 1, j - 1)
    print X[i]
else if c[i - 1, j] >= c[i, j - 1]
    PRINT-LCS(c, X, Y, i - 1, j)
else
    PRINT-LCS(c, X, Y, i, j - 1)
```

5. Give a memorized version of LCS-LENGTH that runs in $O(mn)$ time.

```
MEMO-LCS-LENGTH(X, Y, i, j)

if c[i, j] > -1
    return c[i, j]

if i == 0 or j == 0
    return c[i, j] = 0

if x[i] == y[j]
    return c[i, j] = LCS-LENGTH(X, Y, i - 1, j - 1) + 1

return c[i, j] = max(LCS-LENGTH(X, Y, i - 1, j), LCS-LENGTH(X, Y, i, j - 1))
```

6. Determine the LCS of (1, 0, 0, 1, 0, 1, 0, 1) and (0, 1, 0, 1, 1, 0, 1, 1, 0)

The LCS of these two sequences is (1, 0, 1, 0, 1, 0)

7. Give an $O(n^2)$ time algorithm to find the longest monotonically increasing subsequence of a sequence of n number!

```
MEMO-LCS-LENGTH(X, Y, i, j)
```

```
if c[i, j] > -1
```

```
    return c[i, j]
```

```
if i == 0 or j == 0
```

```
    return c[i, j] = 0
```

```
if x[i] == y[j]
```

```
    return c[i, j] = LCS-LENGTH(X, Y, i - 1, j - 1) + 1
```

```
return c[i, j] = max(LCS-LENGTH(X, Y, i - 1, j), LCS-LENGTH(X, Y, i, j - 1))
```